JTS Topology Suite

Developer's Guide

Amended for the curve stack on this fork

Base text: Version 1.4 (17 October 2003) · Amendment: 16 August 2026

Against feature/sfa-curve-rgr @ 8e39e39a · PR #7 is the source of truth

This is a JTS guide. The 2003–2005 Vivid Solutions text is kept where it is still true. Sections that would lie about curves, I/O, overlay, Hausdorff, or TestBuilder on this fork are amended against `feature/sfa-curve-rgr` *at* `8e39e39a` *(PR #7).*

LocationTech JTS remains the upstream project. This fork is not described here as if it were already upstream. Package names in this tree are `org.locationtech.jts`*; the 2003–2005 copies used* `com.vividsolutions.jts`*.*

Document change control

Revision	Date	Author	Change
1.4	17 Oct 2003	Jonathan Aquino	Initial draft, created for JTS 1.4.
1.4-fork	16 Aug 2026	This fork	Amend stale curve / I/O / overlay / Hausdorff claims against #7 @ 8e39e39a. Keep JTS guide voice. Do not rebrand as LocationTech upstream.

Honesty lock (this amendment)

• Verified against the #7 tree at 8e39e39a. Unmerged drafts are not described as shipped. This document does not restack onto other bars.

• Types named below are the ones on #7. Nothing else is invented.

• OverlayNGCurve, never OverlayNGCurved. No public noder.

• License remains the JTS dual licence: Eclipse Public License 2.0 and Eclipse Distribution License 1.0. See LICENSE_EPLv2.txt, LICENSE_EDLv1.txt, and LICENSES.md. The 2005 TestBuilder guide's ACME licence appendix is historical and is not the project licence.

Table of contents

1. Overview

The JTS Topology Suite is a Java API that implements a core set of spatial data operations using an explicit precision model and robust geometric algorithms. The 2003 guide described a complete model for specifying 2-D *linear* Geometry. That sentence is still the right description of JTS core. This fork adds opt-in SQL-MM curve types in a separate module; it does not replace the linear model.

JTS is intended to be used in the development of applications that support the validation, cleaning, integration and querying of spatial datasets. This document is for developers who want to use the API. It keeps the 2003 walkthrough (WKT, factory, relate, overlay, buffer, polygonize, line merge, custom coordinates) and amends the places that would now lie.

1.1 Other resources

• OpenGIS Simple Features Specification for SQL Revision 1.1 (SFS) — the linear spatial model and predicates JTS implements.

• OGC Simple Features Access 1.2.1 / ISO 19125-2 — the SQL-MM curve keywords and WKB codes 8–12 used by the opt-in curve module.

• JTS Technical Specifications (the sibling PDF in this directory) — design specification, now amended for the same tree.

• JTS JavaDoc for packages under `org.locationtech.jts`.

> *The 2003 note "this document is under construction" is left as history. The amendment is the curve / I/O / overlay / Hausdorff honesty, not a rewrite of the linear API.*

2. Getting started

The most common JTS tasks involve creating and using Geometry objects. The easiest way to create a linear Geometry by hand is still a WKTReader:

```
Geometry g1 = new WKTReader().read(
    "LINESTRING (0 0, 10 10, 20 20)");
```

In a program it is usually easier to use a GeometryFactory:

```
Coordinate[] coordinates = new Coordinate[] {
    new Coordinate(0, 0), new Coordinate(10, 10),
    new Coordinate(20, 20) };
Geometry g1 = new GeometryFactory().createLineString(coordinates);
```

Once you have a Geometry you can compute intersection, area, envelope, centroid, and buffer. See the JavaDoc for `org.locationtech.jts.geom.Geometry`.

> **Amendment — creating a curve**
>
> • On this fork the concrete SFA/SQL-MM curve types live in the `jts-curve` module (`org.locationtech.jts.geom.curve`): **CircularString**, **CompoundCurve**, **CurvePolygon**, and the collections **MultiCurve** and **MultiSurface**. Construction is opt-in: the default `GeometryFactory` throws on `createCircularString` / `createCompoundCurve` / `createCurvePolygon` / `createMultiCurve` / `createMultiSurface`. Use `CurveGeometryFactory`. Constructors do not linearise; there is no Linearizable-on-construct.
>
> • A core WKTReader still throws ParseException on CIRCULARSTRING. Opt in:

```
CurveGeometryFactory gf = new CurveGeometryFactory();
CurveWKTReader rdr = new CurveWKTReader(gf);
Geometry arc = rdr.read("CIRCULARSTRING (0 0, 5 5, 10 0)");
// arc.getGeometryType() == "CircularString"
```

3. Computing spatial relationships

JTS follows the Dimensionally Extended 9-Intersection Matrix (DE-9IM) model specified by the OGC. To compute the DE-9IM for two Geometries:

```
Geometry a = /* ... */;
Geometry b = /* ... */;
IntersectionMatrix m = a.relate(b);
```

Most relationships of interest can be specified as a pattern which matches a set of intersection matrices. JTS also provides boolean predicates:

Method	Meaning
Equals	The Geometries are topologically equal.
Disjoint	The Geometries have no point in common.
Intersects	The Geometries have at least one point in common (inverse of Disjoint).
Touches	The Geometries have at least one boundary point in common, but no interior points.
Crosses	The Geometries share some but not all interior points, and the intersection dimension is less than that of at least one operand.
Within	Geometry A lies in the interior of Geometry B.
Contains	Geometry B lies in the interior of Geometry A (inverse of Within).
Overlaps	The Geometries share some but not all points, and the intersection has the same dimension as the Geometries.

In some cases the precise definition is subtle. Refer to the JTS Technical Specifications. On curve operands, public relate/predicates that have not taken a certified-disc path still see control-point chords unless the caller linearises explicitly via Linearizable.toLinear(tolerance).

4. Computing overlay operations

The 2003 guide illustrated the four set operations and listed Buffer and ConvexHull beside them. The meanings are unchanged for linear Geometry:

Method	Meaning
Buffer	The Polygon or MultiPolygon of all points within a specified distance of the Geometry.
ConvexHull	The smallest convex Polygon that contains all the points in the Geometry.
Intersection	The set of all points which lie in both A and B.
Union	The set of all points which lie in A or B.
Difference	The set of all points which lie in A but not in B.
SymDifference	The set of all points which lie in either A or B but not both.

On linear Geometry the production overlay is OverlayNG (`org.locationtech.jts.operation.overlayng`). The 2003 noding-graph overlay is historical.

Amendment — overlay of curves

• The curve overlay type is **OverlayNGCurve** — never OverlayNGCurved. It lives in `jts-curve` (`org.locationtech.jts.operation.overlayng.curve`). There is no public noder. Closed-form answers are limited to certified discs. Anything else densifies and delegates to core OverlayNG. Geometry.`intersection` / `union` / `difference` / `symDifference` on a curve type are not a general circular overlay.

• Closed-form constructions on this tree are certified discs and the single-arc hull. Disc buffer of a full circular disc is a disc of radius r+d (or empty if r+d ≤ 0). An open arc stays on BufferOp chords. Anything that is not a certified disc or a single arc uses `toLinear` and the linear algorithm — a fallback, not a fail, and not a closed form.

```
import org.locationtech.jts.operation.overlayng.curve.OverlayNGCurve;
Geometry cap = OverlayNGCurve.intersection(a, b);
Geometry cup = OverlayNGCurve.union(a, b);
Geometry sub = OverlayNGCurve.difference(a, b);
Geometry xor = OverlayNGCurve.symDifference(a, b);
```

5. Computing buffers

In GIS, buffering computes the area containing all points within a given distance of a Geometry (the Minkowski sum with a disc). Positive and negative buffers are sometimes called dilation and erosion. In CAD/CAM the outline is an offset curve.

You can compute a buffer with Geometry.buffer or BufferOp. The input may be of any type, including GeometryCollections. The result is always an area type (Polygon or MultiPolygon) and may be empty (for example a negative buffer of a LineString).

Positive buffers contain the input. Negative buffers are contained in the input. A negative buffer of a LineString or Point is empty. A zero-distance buffer can be used as an efficient union of polygons.

5.1 Basic buffering

```
Geometry g = /* ... */;
Geometry buffer = g.buffer(100.0);
```

5.2 End cap styles

Buffer polygons can be computed with different line end-cap styles via BufferOp / BufferParameters:

Style	Description
CAP_ROUND	The usual round end caps.
CAP_FLAT / CAP_BUTT	End caps truncated flat at the line ends (the 2003 name was CAP_BUTT).
CAP_SQUARE	End caps squared off at the buffer distance beyond the line ends.

```
BufferOp bufOp = new BufferOp(g);
bufOp.setEndCapStyle(BufferParameters.CAP_FLAT);
Geometry buffer = bufOp.getResultGeometry(distance);
```

5.3 Approximation quantization

The exact buffer outline of linear Geometry usually contains circular sections. JTS approximates those with line segments. The degree of approximation is the number of quadrant segments used for a quarter-circle. The default is 8 (less than 2% maximum distance error). A value of 12 reduces that to less than 1%.

```
Geometry buffer = g.buffer(100.0, 16);
```

Amendment — buffer of curves

• A full circular disc (certified CurvePolygon) has a closed-form buffer: another disc of radius r+d, or empty if r+d ≤ 0.

• An open CircularString is not a disc. Its buffer is BufferOp on chords. That is not a closed form.

6. Polygonization

Polygonization forms polygons from linework that encloses areas. The linework must be fully noded: LineStrings must not cross and may touch only at endpoints. Polygonizer takes a set of fully noded LineStrings and forms the enclosed polygons. Dangles and cut edges can be reported.

```
Polygonizer polygonizer = new Polygonizer();
polygonizer.add(lines);
Collection polys   = polygonizer.getPolygons();
Collection dangles = polygonizer.getDangles();
Collection cuts    = polygonizer.getCutEdges();
```

If the set of lines is not correctly noded the Polygonizer will still operate, but the result will not be valid. See §9.1 for noding.

7. Merging a set of LineStrings

A spatial operation such as union can produce chains of small LineStrings. LineMerger sews these together. It assumes the input is noded (LineStrings do not cross; only endpoints may touch). The output is also noded. If directions disagree, the result follows the majority.

```
LineMerger lineMerger = new LineMerger();
lineMerger.add(lineStrings);
Collection merged = lineMerger.getMergedLineStrings();
```

8. Using custom CoordinateSequences

By default JTS uses arrays of Coordinates. You may want a more compact sequence, or extra information on each coordinate (measures for linear referencing). Implement CoordinateSequence and CoordinateSequenceFactory, then construct a GeometryFactory parameterized by that factory. All Geometries created from it will use your sequence.

If the sequence is not based on an array of standard Coordinates, there may be a small marshalling cost. Curve types use the same factory hook; CurveGeometryFactory can be constructed with a custom CoordinateSequenceFactory.

9. Tips and techniques

9.1 Noding a set of LineStrings

Many operations assume input is noded: LineStrings never cross. A simple trick is to union them; the union process nodes the LineStrings. Unary union / OverlayNG is the current production path for many polygons; the 2003 "buffer of a GeometryCollection at distance 0" trick still works but is no longer the first recommendation.

9.2 Unioning many polygons

Repeated Polygon.union is correct but slow. Prefer UnaryUnionOp / OverlayNG.union on a collection. The 2003 GeometryCollection.buffer(0) trick remains documented as history.

10. Curve types on this fork (amendment)

On this fork the concrete SFA/SQL-MM curve types live in the `jts-curve` module (`org.locationtech.jts.geom.curve`): **CircularString**, **CompoundCurve**, **CurvePolygon**, and the collections **MultiCurve** and **MultiSurface**. Construction is opt-in: the default `GeometryFactory` throws on `createCircularString` / `createCompoundCurve` / `createCurvePolygon` / `createMultiCurve` / `createMultiSurface`. Use `CurveGeometryFactory`. Constructors do not linearise; there is no Linearizable-on-construct.

WKT keyword	Java class	Extends
CIRCULARSTRING	org.locationtech.jts.geom.curve.CircularString	LineString
COMPOUNDCURVE	org.locationtech.jts.geom.curve.CompoundCurve	LineString
CURVEPOLYGON	org.locationtech.jts.geom.curve.CurvePolygon	Polygon
MULTICURVE	org.locationtech.jts.geom.curve.MultiCurve	MultiLineString
MULTISURFACE	org.locationtech.jts.geom.curve.MultiSurface	MultiPolygon

Maven artifact: `org.locationtech.jts:jts-curve`. Callers instantiate CurveGeometryFactory / CurveWKTReader / CurveWKBWriter explicitly. The module does not auto-register via ServiceLoader.

Linearizable.toLinear(tolerance) is the explicit densify. Spatial methods that have not taken a certified closed form still fall through to the parent linear type or to a linearised copy. That is not hidden linearise-on-construct.

Closed-form constructions on this tree are certified discs and the single-arc hull. Disc buffer of a full circular disc is a disc of radius r+d (or empty if r+d ≤ 0). An open arc stays on BufferOp chords. Anything that is not a certified disc or a single arc uses `toLinear` and the linear algorithm — a fallback, not a fail, and not a closed form.

The curve overlay type is **OverlayNGCurve** — never OverlayNGCurved. It lives in `jts-curve` (`org.locationtech.jts.operation.overlayng.curve`). There is no public noder. Closed-form answers are limited to certified discs. Anything else densifies and delegates to core OverlayNG. Geometry.`intersection` / `union` / `difference` / `symDifference` on a curve type are not a general circular overlay.

11. WKT and WKB on this fork (amendment)

Core `WKTReader` still rejects `CIRCULARSTRING` and the other SQL-MM keywords. Read them with `CurveWKTReader`. Write structured members with `CurveWKTWriter`. Core `WKTWriter` already emits the subclass keyword via `getGeometryType()` for a flat CircularString; composite types need the curve writer to keep member tags.

WKB type codes 8–12 are first-class in core `WKBReader` and `WKBConstants` (CircularString=8, CompoundCurve=9, CurvePolygon=10, MultiCurve=11, MultiSurface=12). Construction still requires a curve-capable factory; otherwise the reader throws. `CurveWKBWriter` emits codes 8–12. A bare `WKBWriter` still walks the parent `instanceof` ladder and writes a CircularString as LineString (code 2) and a CurvePolygon as Polygon (code 3). That is what a direct `WKBWriter.write` call does on this tree.

```
// Read codes 8–12 as curve types:
Geometry g = new WKBReader(new CurveGeometryFactory()).read(bytes);
// or the no-arg convenience:
Geometry g2 = new CurveWKBReader().read(bytes);

// Write codes 8–12:
byte[] wkb = new CurveWKBWriter().write(g);

// Bare writer still emits 2 / 3 for the parent types:
byte[] flat = new WKBWriter().write(g);  // CircularString → type 2
```

12. Hausdorff on this fork (amendment)

Public `DiscreteHausdorffDistance` on this tree (commit `0ca71b`, merged on #7) has a closed form for **two pairs only**: (1) a single-arc CircularString toward a single-segment LineString, apex √949/6 – 7/6 ≈ 3.967641 for the witness `CIRCULARSTRING (0 0, 2 3, 10 0)` vs `LINESTRING (0 0, 10 0)`; (2) two circular discs (full-circle CurvePolygon). A MultiSurface unwrap of one disc is the same disc pair. The exact path skips densify; densify is not the laser. For every other pair the public API still sees vertices / chords. Tests keep `fail()`. This document does not claim DirectedHausdorffDistance replace, port, or ship. Fréchet remains open.

The 2003 Developer's Guide did not document Hausdorff. It is recorded here so a reader of this fork is not left with the public-API chord behaviour as an unspoken surprise.

13. Licence

License remains the JTS dual licence: Eclipse Public License 2.0 and Eclipse Distribution License 1.0. See LICENSE_EPLv2.txt, LICENSE_EDLv1.txt, and LICENSES.md. The 2005 TestBuilder guide's ACME licence appendix is historical and is not the project licence.

End of Developer's Guide amendment. Figures from the 2003 Amyuni conversion (overlay cartoon, end-cap styles, quadrant-segment diagrams) are omitted; the prose they illustrated is kept.